SGHR与MIRETR融合实现多实例修正多视角配准详细步骤

一、环境搭建

1. 统一环境创建

创建同时兼容 SGHR 和 MIRETR 的 conda 环境，两个项目均基于 Python3.7 开发，环境配置如下：

Code block

```
conda create -n multi_instance_multiview python=3.7
conda activate multi_instance_multiview
```

2. 依赖安装

依次安装两个项目的所有依赖，解决潜在的依赖冲突：

Code block

```
# 安装Pytorch基础环境（适配两个项目的版本要求）
conda install pytorch==1.7.1 torchvision==0.8.2 torchaudio==0.7.2 cudatoolkit=11.0 -c pytorch

# 安装SGHR所需依赖
pip install open3d==0.8.0.0
# 安装MinkowskiEngine用于FCGF/YOHO特征提取
conda install openblas-devel -c anaconda
git clone https://github.com/NVIDIA/MinkowskiEngine.git
cd MinkowskiEngine
python setup.py install --blas_include_dirs=${CONDA_PREFIX}/include --blas=openblas

# 安装MIRETR所需依赖
cd path/to/MIRETR
pip install -r requirements.txt
python setup.py build develop
```

二、项目代码整合

1. 目录结构规划

将两个项目的代码整合到统一的工作目录，同时新增融合模块的目录，结构如下：

Code block

```
multi_instance_multiview/
├── SGHR/           # SGHR原始代码
│   ├── TransSync/          # 姿态同步与IRLS优化模块
│   ├── checkpoints/        # 预训练模型权重
│   ├── data/               # 数据集目录
│   ├── dataops/            # 数据操作工具
│   ├── model/              # 姿态图构建与特征提取模型
│   ├── train/              # 训练脚本
│   ├── utils/              # 通用工具
│   ├── yoho/               # YOHO特征提取模块
│   ├── Test.py             # 原始测试脚本
│   ├── Train.py            # 原始训练脚本
│   └── demo.py             # 原始演示脚本
├── MIRETR/         # MIRETR原始代码
│   ├── release/            # 训练与测试脚本
│   ├── vision3d/           # 3D视觉基础库
│   ├── data/               # 数据集目录
│   ├── requirements.txt    # 依赖清单
│   └── setup.py            # 安装脚本
├── fusion/         # 融合后的自定义模块
│   ├── preprocess.py       # 整合预处理与实例分割
│   ├── instance_pose.py    # 实例级初始配准
│   ├── pose_graph.py       # 实例感知姿态图构建
│   ├── irls_optimize.py    # 实例感知IRLS优化
│   ├── global_optimize.py  # 实例约束全局优化
│   └── main.py             # 融合框架主入口
```

2. 预处理模块融合

将 MIRETR 的实例感知 Transformer 模块集成到 SGHR 的预处理流程中，在 `fusion/preprocess.py` 中实现：

Code block

```python
import torch
import open3d as o3d
from SGHR.yoho.extract import extract_yoho_features
from MIRETR.vision3d.models import InstanceAwareTransformer

def preprocess_point_clouds(point_cloud_paths, miretr_ckpt_path):
```

```
    # 加载MIRETR实例分割预训练模型
    instance_model = InstanceAwareTransformer.load_from_checkpoint(miretr_ckpt_path)
    instance_model.eval()

    processed_clouds = []
    for path in point_cloud_paths:
        # 读取点云
        pcd = o3d.io.read_point_cloud(path)
        points = torch.tensor(pcd.points, dtype=torch.float32)

        # SGHR原始预处理：下采样与YOHO特征提取
        downsampled = pcd.voxel_down_sample(voxel_size=0.05)
        yoho_feats = extract_yoho_features(downsampled)

        # MIRETR实例分割：提取实例掩码与实例特征
        with torch.no_grad():
            instance_masks, instance_feats = instance_model(points.unsqueeze(0))

        # 保存处理后的点云数据
        processed_clouds.append({
            "point_cloud": downsampled,
            "yoho_features": yoho_feats,
            "instance_masks": instance_masks[0],
            "instance_features": instance_feats[0]
        })
    return processed_clouds
```

3. 实例感知姿态图构建

修改 SGHR 的姿态图构建逻辑，加入实例约束，在 `fusion/pose_graph.py` 中实现：

Code block

```
import numpy as np
from SGHR.model.global_feature import compute_global_feature

def build_instance_aware_graph(processed_clouds, top_k=5):
    # 1. 计算所有视角的全局特征，用于估计视角重叠度
    global_features = []
    for cloud in processed_clouds:
        global_feat = compute_global_feature(cloud["yoho_features"])
        global_features.append(global_feat)

    # 2. 构建视角间边：过滤跨实例的无效边
    view_edges = []
```

```python
    for i in range(len(processed_clouds)):
        # 计算与其他视角的重叠度
        overlaps = [(np.dot(global_features[i], global_features[j]) + 1)/2 for j in range(len(processed_clouds))]
        # 选择前k个高重叠度的视角
        top_indices = np.argsort(overlaps)[-top_k:][::-1]

        for j in top_indices:
            if i == j:
                continue
            # 检查两个视角的实例匹配度
            instance_match = check_instance_overlap(processed_clouds[i]["instance_masks"], processed_clouds[j]["instance_masks"])
            if instance_match > 0.3:  # 实例匹配度阈值
                # 边权重 = 视角重叠度 * 实例匹配度
                weight = overlaps[j] * instance_match
                view_edges.append((i, j, weight))

    # 3. 构建实例间边：同一实例在不同视角的对应边
    instance_edges = []
    # 先统计所有实例ID
    all_instances = set()
    for cloud in processed_clouds:
        all_instances.update(torch.unique(cloud["instance_masks"]).numpy())

    for instance_id in all_instances:
        # 找到包含该实例的所有视角
        instance_views = []
        for idx, cloud in enumerate(processed_clouds):
            if instance_id in cloud["instance_masks"]:
                instance_views.append(idx)

        # 为这些视角构建实例边
        for a in range(len(instance_views)):
            for b in range(a+1, len(instance_views)):
                i = instance_views[a]
                j = instance_views[b]
                # 计算该实例在两个视角的相对姿态
                relative_pose = compute_instance_relative_pose(processed_clouds[i], processed_clouds[j], instance_id)
                # 边权重为实例配准的内点比例
                inlier_ratio = get_instance_inlier_ratio(processed_clouds[i], processed_clouds[j], instance_id)
                instance_edges.append((i, j, inlier_ratio, relative_pose))

    return view_edges, instance_edges
```

```python

def check_instance_overlap(mask1, mask2):
    # 计算两个视角的实例掩码交集比例
    mask1_np = mask1.numpy()
    mask2_np = mask2.numpy()
    # 统计共同实例的点数量
    common_instances = set(mask1_np) & set(mask2_np)
    if not common_instances:
        return 0.0
    # 计算重叠比例
    count1 = np.sum([np.sum(mask1_np == c) for c in common_instances])
    count2 = np.sum([np.sum(mask2_np == c) for c in common_instances])
    return min(count1/len(mask1_np), count2/len(mask2_np))
```

4. 实例感知 IRLS 优化

修改 SGHR 的 IRLS 优化逻辑，加入实例一致性权重更新，在 `fusion/irls_optimize.py` 中实现：

Code block

```python
from SGHR.TransSync.sync import rotation_sync, translation_sync

def instance_aware_irls(processed_clouds, view_edges, instance_edges, max_iter=20, alpha=0.5, sigma=0.1):
    # 初始化视角姿态与实例姿态
    init_poses = [np.eye(4) for _ in range(len(processed_clouds))]
    instance_poses = {}
    for edge in instance_edges:
        i, j, weight, rel_pose = edge
        if edge not in instance_poses:
            instance_poses[(i,j)] = rel_pose

    # 迭代优化
    for iter in range(max_iter):
        # 1. 姿态同步：同时优化视角姿态与实例姿态
        # 旋转同步
        view_rotations = [pose[:3,:3] for pose in init_poses]
        instance_rotations = [rel_pose[:3,:3] for rel_pose in instance_poses.values()]
        refined_view_rots, refined_instance_rots = rotation_sync(view_edges, instance_edges, view_rotations, instance_rotations)

        # 平移同步
        view_trans = [pose[:3,3] for pose in init_poses]
```

```python
        instance_trans = [rel_pose[:3,3] for rel_pose in instance_poses.values()]
        refined_view_trans, refined_instance_trans = translation_sync(view_edges, instance_edges, refined_view_rots, refined_instance_rots, view_trans, instance_trans)

        # 2. 更新权重：结合视角残差与实例残差
        new_view_weights = []
        for edge in view_edges:
            i, j, old_weight = edge
            # 计算视角姿态残差
            view_res = compute_pose_residual(init_poses[i], init_poses[j], edge)
            # 计算实例姿态残差
            instance_res = compute_instance_residual(instance_poses.get((i,j), np.eye(4)), refined_instance_rots[i], refined_instance_rots[j])
            # 总残差
            total_res = view_res + alpha * instance_res
            # 更新权重
            new_weight = old_weight * np.exp(-total_res**2 / (2 * sigma**2))
            new_view_weights.append(new_weight)

        # 3. 更新姿态
        for i in range(len(init_poses)):
            init_poses[i][:3,:3] = refined_view_rots[i]
            init_poses[i][:3,3] = refined_view_trans[i]

        # 4. 检查收敛
        pose_change = np.mean([np.linalg.norm(init_poses[i] - old_pose) for i, old_pose in enumerate(init_poses)])
        if pose_change < 1e-6:
            break

    return init_poses, instance_poses

def compute_pose_residual(pose1, pose2, relative_pose):
    # 计算两个姿态的残差
    est_rel = np.linalg.inv(pose1) @ pose2
    return np.linalg.norm(est_rel - relative_pose)

def compute_instance_residual(instance_rel, rot1, rot2):
    # 计算实例姿态的残差
    est_rel_rot = rot1.T @ rot2
    return np.linalg.norm(est_rel_rot - instance_rel[:3,:3])
```

5. 全局优化模块

实现实例约束的全局优化，确保实例在多视角中的姿态一致，在 `fusion/global_optimize.py` 中实现：

Code block

```
import numpy as np

def global_instance_optimize(processed_clouds, view_poses, instance_poses):
    # 1. 收集每个实例在所有视角中的姿态
    instance_global_poses = {}
    all_instances = set()
    for cloud in processed_clouds:
        all_instances.update(torch.unique(cloud["instance_masks"]).numpy())

    for instance_id in all_instances:
        instance_poses_list = []
        for idx, cloud in enumerate(processed_clouds):
            if instance_id in cloud["instance_masks"]:
                # 计算该实例的全局姿态：视角姿态 * 实例相对姿态
                rel_pose = instance_poses.get((idx, idx), np.eye(4))  # 自身相对姿态为单位矩阵
                global_pose = view_poses[idx] @ rel_pose
                instance_poses_list.append(global_pose)

        # 计算实例的平均全局姿态
        if instance_poses_list:
            avg_pose = np.mean(instance_poses_list, axis=0)
            instance_global_poses[instance_id] = avg_pose

    # 2. 更新视角姿态，确保与实例全局姿态一致
    refined_view_poses = []
    for idx, pose in enumerate(view_poses):
        cloud = processed_clouds[idx]
        # 计算该视角下所有实例的期望姿态
        expected_poses = []
        for instance_id in torch.unique(cloud["instance_masks"]).numpy():
            if instance_id in instance_global_poses:
                # 期望的实例相对姿态 = 视角姿态逆 * 实例全局姿态
                expected_rel = np.linalg.inv(pose) @ instance_global_poses[instance_id]
                expected_poses.append(expected_rel)

        # 微调视角姿态，最小化与所有实例期望姿态的残差
        if expected_poses:
            # 加权平均微调
            avg_rel = np.mean(expected_poses, axis=0)
            refined_pose = pose @ avg_rel
```

```
            refined_view_poses.append(refined_pose)
        else:
            refined_view_poses.append(pose)

    return refined_view_poses, instance_global_poses
```

6. 融合框架主入口

在 `fusion/main.py` 中整合所有模块：

Code block

```
import argparse
from preprocess import preprocess_point_clouds
from pose_graph import build_instance_aware_graph
from irls_optimize import instance_aware_irls
from global_optimize import global_instance_optimize
import open3d as o3d

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--data_path", type=str, required=True, help="Path to point cloud dataset")
    parser.add_argument("--miretr_ckpt", type=str, required=True, help="Path to MIRETR pretrained checkpoint")
    parser.add_argument("--top_k", type=int, default=5, help="Top k views for pose graph")
    parser.add_argument("--max_iter", type=int, default=20, help="Max IRLS iterations")
    args = parser.parse_args()

    # 1. 预处理点云
    import os
    point_cloud_paths = [os.path.join(args.data_path, f) for f in os.listdir(args.data_path) if f.endswith(".ply")]
    processed_clouds = preprocess_point_clouds(point_cloud_paths, args.miretr_ckpt)

    # 2. 构建实例感知姿态图
    view_edges, instance_edges = build_instance_aware_graph(processed_clouds, args.top_k)

    # 3. 实例感知IRLS优化
    view_poses, instance_poses = instance_aware_irls(processed_clouds, view_edges, instance_edges, args.max_iter)

    # 4. 全局优化
```

```
    refined_view_poses, instance_global_poses = global_instance_optimize(processed_clouds, view_poses, instance_poses)

    # 5. 保存与可视化结果
    # 保存配准后的点云
    for idx, (cloud, pose) in enumerate(zip(processed_clouds, refined_view_poses)):
        transformed_pcd = cloud["point_cloud"].transform(pose)
        o3d.io.write_point_cloud(f"registered_cloud_{idx}.ply", transformed_pcd)

    # 保存实例姿态
    with open("instance_poses.txt", "w") as f:
        for instance_id, pose in instance_global_poses.items():
            f.write(f"Instance {instance_id}:\n")
            f.write(f"{pose}\n\n")

    # 可视化结果
    vis = o3d.visualization.Visualizer()
    vis.create_window()
    for cloud in processed_clouds:
        vis.add_geometry(cloud["point_cloud"])
    vis.run()
    vis.destroy_window()

if __name__ == "__main__":
    main()
```

三、测试验证

1. 数据集准备

选择 ROBI 数据集进行测试，该数据集包含多实例的复杂场景，同时适配两个项目的要求：

1. 下载 ROBI 数据集：https://github.com/zhiyuanYU134/ROBI_Toolkit
2. 按照 SGHR 的要求预处理数据集，生成 YOHO 特征文件
3. 按照 MIRETR 的要求预处理数据集，生成实例掩码文件

2. 测试命令

运行融合后的框架进行测试：

Code block

```
cd fusion
```

```
python main.py --data_path ../data/robi/scenes --miretr_ckpt
../MIRETR/pretrained/robi.ckpt --top_k 5 --max_iter 20
```

3. 评价指标

- **多视角配准精度**：计算配准后所有点云的平均距离误差（Registration Error）
- **实例姿态精度**：使用 ADD（Average Distance of Model Points）指标评价每个实例的姿态精度
- **效率**：记录配准的总时间，对比原始 SGHR 和 MIRETR 的效率

4. 对比实验

1. 与原始 SGHR 对比：验证多实例修正后配准精度的提升
2. 与原始 MIRETR 对比：验证多视角配准全局一致性的提升
3. 消融实验：验证实例感知模块、实例约束模块的贡献

四、常见问题解决

1. **依赖冲突**：如果 Open3d 版本冲突，尝试使用兼容的版本，或者修改代码适配不同的版本
2. **模型加载失败**：确保预训练模型的路径正确，同时检查模型的输入输出维度是否匹配
3. **内存溢出**：多实例和多视角的点云数据量较大，可使用更小的 batch size，或者使用更高效的特征提取方法
4. **配准收敛缓慢**：调整 IRLS 的迭代次数、权重更新的参数（alpha、sigma），优化姿态同步的算法

（注：文档部分内容可能由 AI 生成）